Generative-AI–Based Automated Analysis and Risk Scoring of Fraudulent APKs

Datasets and Solution Approach

Team Chernobyl

Bismay Samal
Mohit Ranjan Naik
Amiya Behera
Shreya Mishra

Abstract

Mobile-banking fraud increasingly relies on malicious Android application packages (APKs) that impersonate legitimate banking and payment applications to steal one-time passwords, harvest credentials, abuse accessibility services, and exfiltrate sensitive data. Manual analysis of such samples is accurate but slow and does not scale with submission volume. This document specifies the datasets we will use to train and validate the machine-learning component of an automated APK risk-assessment platform, and the layered solution approach we will follow. The approach fuses transparent rule-based scoring, supervised machine learning, controlled dynamic behaviour analysis, and a generative-AI reasoning layer to produce explainable risk scores and analyst-ready investigation reports, with explicit attention to concept drift, false-positive control, and rigorous, honest evaluation.

1 Introduction

Android remains the dominant mobile platform and, consequently, the primary target of mobile-banking fraud. Malicious APKs increasingly impersonate legitimate banking, payment, and utility applications to steal one-time passwords (OTPs), harvest credentials, abuse accessibility services, and exfiltrate sensitive data. Manual triage of such samples is slow and does not scale with the volume of submissions a fraud or Security Operations Centre (SOC) team receives.

This document describes the *datasets we will use* to train and validate the machine-learning component of our platform, and the *solution approach* we will follow to deliver automated, explainable risk assessment of suspicious APKs. Our system combines transparent rule-based scoring, supervised machine learning, controlled dynamic behaviour analysis, and a generative-AI reasoning layer that produces human-readable investigation reports.

1.1 Objectives

Our objectives are to: (i) extract security-relevant evidence from an APK through static and (optionally) dynamic analysis; (ii) produce a calibrated, explainable risk score that discriminates malicious from benign applications while keeping false positives low; (iii) train a machine-learning classifier that generalises to recent, previously unseen samples; and (iv) generate human-readable investigation reports that communicate findings and recommended actions to both technical and non-technical stakeholders.



Figure 1: Introduction overview illustrating the mobile fraud threat landscape, challenges of manual APK investigation, and the motivation for an automated AI-powered APK risk assessment platform.

1.2 Design principles

Three principles guide the design. *Explainability:* every risk contribution must be traceable to concrete evidence. *Safety:* potentially malicious code is executed only inside an isolated, disposable environment, never on analyst infrastructure. *Honesty:* reported confidence reflects the evidence, and known limitations (drift, evasion, unavailable C2) are stated rather than hidden.

2 Background

2.1 Static versus dynamic analysis

Static analysis inspects an application without running it—parsing the manifest, decompiling bytecode, and extracting permissions, API references, strings, and structural signals. It is fast and safe but can be defeated by encryption, packing, and runtime code loading. *Dynamic analysis* executes the application in a controlled environment and observes actual behaviour—network traffic, file activity, and security-sensitive API calls—revealing intent that static analysis cannot, at the cost of greater complexity and the need for isolation. The two are complementary, and our approach uses both.

2.2 Machine learning for malware detection

Supervised learning over engineered features (permissions, API usage, intents, network indicators) is a well-established approach to Android-malware detection. Its principal challenges are *class imbalance* (benign samples dominate), *concept drift* (malware evolves, degrading models trained on older data), and *evaluation inflation* (near-duplicate samples leaking between train and test). Our methodology explicitly addresses all three.

2.3 Generative AI as a reasoning layer

Large language models excel at correlating heterogeneous evidence and producing fluent, structured explanations. In our design, generative AI does *not* act as a detector; it reasons over already-extracted findings to produce summaries, attack-chain narratives, and recommendations, and to contextualise ambiguous evidence—for example, distinguishing benign third-party library usage from genuinely malicious behaviour.

3 Datasets

The machine-learning model will be trained, validated, and stress-tested on a combination of established Android-malware corpora and curated benign application sets. We deliberately combine *older, well-labelled* datasets (for breadth of malware families and reproducibility) with *recent* datasets and live sampling (to mitigate concept drift, since malware behaviour evolves over time).

3.1 Primary training and benchmark datasets

- **Drebin.** A widely cited Android-malware benchmark containing approximately **5,560 malicious** samples spanning **179 malware families**, alongside a large set of benign applications. Crucially, Drebin provides *pre-extracted static feature sets* per sample—requested and used permissions, API calls, intent actions, hardware components, application components, and embedded network indicators—together with a family-label mapping. These ready-made feature vectors allow us to train and benchmark classifiers without reprocessing raw APKs, making it our primary baseline dataset.
- **CICMalDroid 2020.** A modern dataset of roughly **17,000** Android samples organised into five classes: *Adware*, *Banking malware*, *SMS malware*, *Riskware*, and *Benign*. It includes both static attributes and dynamically observed behaviour (system calls, binder activity, composite behaviours). Its banking- and SMS-malware categories align directly with our fraud-triage objective, and its recency helps the model generalise beyond the older Drebin distribution.
- **CIC-AndMal2017.** A dataset emphasising *dynamic and network-traffic* features, comprising several hundred malware samples across major families (ransomware, scareware, SMS malware, adware) and several thousand benign applications. We will use it to enrich the feature space with runtime and network-flow characteristics that static analysis cannot capture.
- **AndroZoo.** A continuously growing repository of **millions** of real-world APKs collected from Google Play and numerous third-party markets, each annotated with multi-engine anti-virus detection counts and collection metadata. We will use AndroZoo as a source of *fresh, diverse* benign and malicious samples drawn from recent time windows, specifically to build an out-of-distribution evaluation set for measuring concept drift and real-world false-positive rates.
- **Android Malware Genome Project (MalGenome).** A historical reference set of **1,260** malware samples across **49 families**. Although dated, it provides a clean, well-documented baseline of classic malware behaviours that is useful for sanity-checking detectors and for family-level analysis.
- **KronoDroid.** A hybrid dataset that records *timestamped* samples with both static and dynamic features captured on real devices and emulators. Its temporal labelling makes it particularly valuable for time-aware evaluation, i.e. training on earlier samples and testing on later ones to simulate realistic deployment and quantify drift.

3.2 Benign application sources

A robust classifier requires a representative, modern benign distribution to keep false-positive rates low. In addition to the benign partitions of the datasets above, we will curate benign applications from:

- **F-Droid** — a catalogue of free and open-source Android applications, useful because the sources are auditable and the apps are demonstrably non-malicious.
- **Google Play (top and long-tail apps via AndroZoo)** — to capture the permission and API profiles of mainstream commercial applications, including advertisement- and analytics-heavy apps that are benign but superficially "suspicious."

3.3 Threat-intelligence corroboration source

For ground-truth verification and IOC enrichment (not as a training feature), we will consult **VirusTotal**, which aggregates verdicts from many anti-malware engines. This supports label validation and provides an independent second opinion during evaluation.

3.4 Feature taxonomy

Features are grouped into static and dynamic families, as summarised in Table 1. We prioritise features expressed *consistently* across data sources, so that a model trained offline applies directly to live submissions with minimal train/serve skew.

Group	Origin	Example features
Permissions	Static	Requested/used permission vectors; counts of high-risk permissions (SMS, contacts, accessibility, overlay, install-packages)
API usage	Static	Presence/frequency of security-relevant API references (SMS sending, accessibility, dynamic code loading, command execution, networking)
Components & intents	Static	Activities/services/receivers/providers; exported components; intent-filter actions
Network indicators	Static	Counts of URLs, IPs, domains; embedded tokens; decoded (Base64/hex) indicators; native-library strings
Obfuscation/packing	Static	Identifier-renaming ratio; multiple DEX files; high-entropy blobs; packer/compiler fingerprints
Certificate	Static	Debug/placeholder identity flags; validity anomalies
Runtime behaviour	Dynamic	System/API calls; started services; granted runtime permissions; file activity
Network flow	Dynamic	Contacted endpoints; protocol/port; traffic-flow statistics

Table 1: Feature taxonomy used for classification.

3.5 Summary of Datasets

Table 2: Datasets used for model training, benchmarking, and evaluation.

Dataset	Approx. Scale	Feature Type	Role in Our Work
Drebin	5.5k malware + benign	Static (permissions, APIs, intents, IOCs)	Primary baseline dataset with pre-extracted features for rapid model training.
CICMalDroid 2020	∼17k	Static + Dynamic	Modern multi-class dataset including banking and SMS malware.
CIC-AndMal2017	∼5.5k	Dynamic + Network	Provides runtime and network-flow feature enrichment.
AndroZoo	Millions	Raw APKs + AV Labels	Fresh real-world APK sampling for concept drift and false-positive evaluation.
MalGenome	1.26k	Static	Historical malware benchmark for family-level validation.
KronoDroid	Tens of Thousands	Hybrid, Timestamped	Supports temporal and concept-drift evaluation.
F-Droid / Google Play	Curated	Raw APKs	Modern benign application distribution.
VirusTotal	On-demand	Multi-engine Verdicts	Label validation, IOC corroboration, and reputation verification.

3.6 Data preparation and partitioning

- **Normalisation and labelling.** Sample identities will be tracked by cryptographic hash; labels will be reconciled across datasets and corroborated with multi-engine verdicts to reduce label noise.
- **Class balance.** Because benign samples vastly outnumber malicious ones, we will apply class weighting and/or balanced resampling to avoid a model biased toward the majority class.
- **Splitting strategy.** We will use stratified train/validation/test splits for in-distribution evaluation and, additionally, a *temporal split* (train on older samples, test on newer ones) to obtain an honest estimate of real-world performance under concept drift.
- **Leakage control.** Family-aware and time-aware splitting will be used to prevent near-duplicate samples from the same campaign appearing in both training and test sets, which would otherwise inflate reported accuracy.

4 Solution Approach

Our approach treats APK risk assessment as a layered pipeline. Each layer contributes independent, explainable evidence, and the layers are fused into a single, transparent risk score accompanied by a natural-language report. Generative AI is used to *reason over and communicate* the extracted evidence, never as the sole detector.

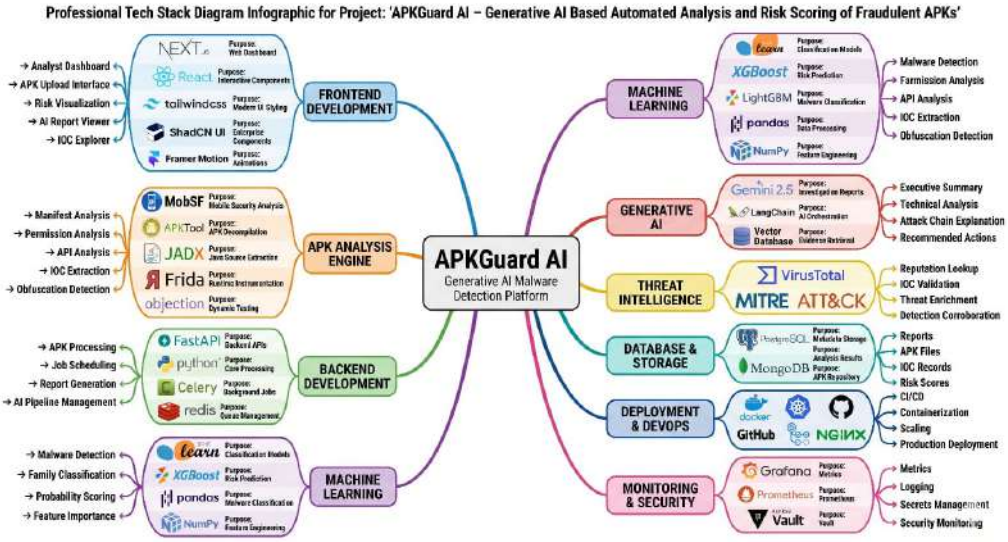


Figure 2: Layered processing pipeline spanning sandbox instrumentation, dynamic behavior tracking, feature extraction, and ingestion into the machine learning input layer.

4.1 Stage 1 — Intake and identity

Each submitted APK will be hashed (e.g. SHA-256/MD5) for de-duplication and blacklisting, and basic metadata (package name, version, target/minimum SDK, file size) will be recorded. Hash-based identity enables consistent labelling and correlation with threat-intelligence feeds.

4.2 Stage 2 — Static analysis

Static analysis inspects the application without executing it and forms the backbone of fast, safe triage. We will extract:

- **Manifest and permissions:** declared components, intent filters, and the set of requested permissions, with emphasis on fraud-relevant permissions (SMS, contacts, accessibility, overlay, install-packages, phone state).
- **Suspicious API usage:** references to APIs associated with OTP theft, accessibility abuse, dynamic code loading, command execution, and network communication, weighted by how discriminative each is in benign versus malicious apps.
- **Indicators of compromise (IOCs):** URLs, IP addresses, domains, embedded tokens, and likely secrets, including indicators recovered by decoding obfuscated (e.g. Base64/hex) strings and by scanning bundled native libraries.
- **Obfuscation and packing signals:** identifier-renaming ratios, multiple DEX files, high-entropy blobs, dynamic loaders, and packer/compiler fingerprints.
- **Signing certificate and attack surface:** certificate red-flags (debug/placeholder identities, implausible validity) and exported components reachable by other applications.

Signature-based screening (rule sets expressing known fraud patterns) and a mapping of findings to a standard adversary-technique taxonomy (MITRE ATT&CK for Mobile) will be applied to make the output actionable for analysts.

A central design concern is *false-positive control*. Many indicators that appear alarming in isolation—requesting network access, referencing cryptography or reflection APIs, or bundling advertising and analytics libraries—are common in entirely legitimate applications. Accordingly,

each static signal is weighted by how *discriminative* it is between benign and malicious applications: rare, high-signal capabilities (programmatic SMS sending, accessibility automation, runtime code loading) carry substantial weight, whereas ubiquitous, low-signal API references are weighted near zero. Standard release- engineering practices (identifier minification, common packers and compilers) are recognised and not penalised as if they were evasion. This calibration is what allows the system to flag genuine fraud while keeping mainstream, advertisement-heavy applications out of the high-severity bands.

4.3 Stage 3 — Controlled dynamic analysis

Because sophisticated samples conceal behaviour from static inspection (e.g. encrypted payloads decrypted only at runtime), we will optionally execute samples inside an *isolated, instrumented sandbox*. Many fraudulent applications ship no launcher icon and instead activate through broadcast receivers (such as device-boot or connectivity events) and background services; a naive "launch the main activity" strategy would therefore observe nothing. To address this, the sandbox systematically *exercises every declared entry point*—the launcher activity, each broadcast receiver (driven with its registered intent actions), and each declared service—and automatically dismisses first-run permission and consent dialogs so that execution is not stalled behind a modal prompt. It then records runtime behaviour: granted permissions, started services, contacted network endpoints, file-system activity, and security-relevant API invocations.

To counter anti-analysis behaviour, the sandbox applies *evasion-resistance*. Many samples fingerprint the environment (device build properties, system properties, telephony identifiers, and the presence of emulator-specific files) and remain dormant when they detect analysis. The sandbox presents realistic device characteristics so that these emulator-detection checks pass and the genuine malicious logic executes. Through dynamic instrumentation it further intercepts security-relevant calls at runtime to *capture decrypted second-stage payloads and command-and-control (C2) endpoints* that never appear in the static image—high-value indicators that would otherwise be invisible. All execution is confined to a disposable, network-restricted, isolated environment with snapshot restoration, and is strictly opt-in; the static pipeline never executes the sample.

4.4 Stage 4 — Feature engineering

Static and dynamic observations will be transformed into a fixed-length feature representation for the classifier. The representation will be drawn from features that are expressed consistently across data sources, including:

- permission presence vectors;
- suspicious-API presence/frequency vectors;
- intent and component indicators;
- counts of network indicators and obfuscation signals;
- selected runtime-behaviour and network-flow features (when dynamic analysis is available).

We will favour features that are robust to superficial obfuscation and that remain comparable between training data and live submissions, in order to minimise feature drift between offline training and deployment.

4.5 Stage 5 — Machine-learning classification

We will train supervised classifiers to estimate the probability that a sample is malicious and, where data permits, its likely family/category (e.g. banking, SMS, adware). Candidate models include:

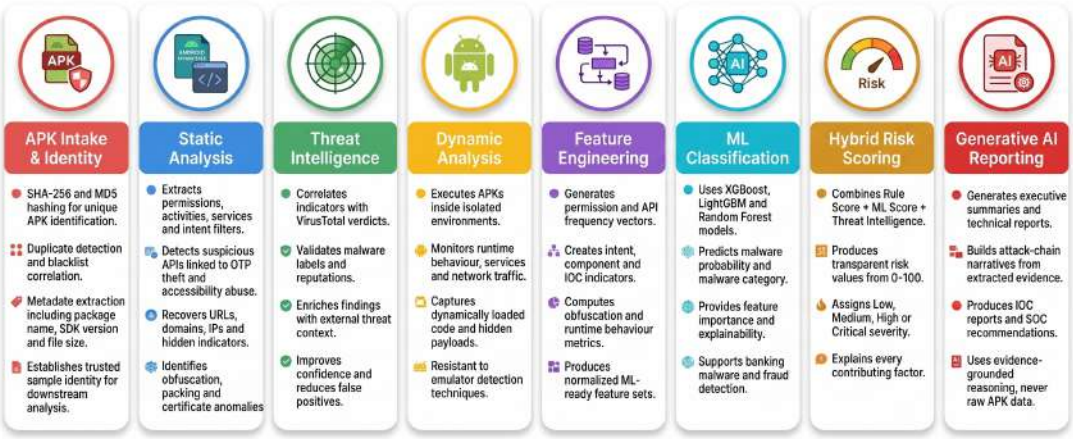


Figure 3: Solution approach workflow of APKGuard AI. The pipeline begins with APK intake and identity management, followed by static and dynamic analysis, feature engineering, machine-learning classification, hybrid risk scoring, threat-intelligence corroboration, and evidence-grounded Generative AI reporting to produce actionable security assessments.

- **Gradient-boosted trees** (e.g. XGBoost/LightGBM) and **Random Forests**, chosen for strong tabular-data performance and interpretability through feature-importance analysis;
- **Logistic Regression** as a transparent baseline;
- Optional advanced directions, such as graph-based models over the application call graph, as a future extension.

The classifier output is treated as one *corroborating signal* fused with the rule-based score—not as a stand-alone verdict—so that the final assessment remains explainable.

Training follows a disciplined protocol. Model selection and hyperparameter tuning use cross-validation on the training partition only; the held-out and temporal test sets are touched once, at the end, to obtain an unbiased estimate. Class imbalance is handled with class weighting and/or balanced resampling, and predicted probabilities are *calibrated* (e.g. via isotonic or Platt scaling) so that a reported probability of, say, 0.8 corresponds to a genuine 80% likelihood—a prerequisite for fusing the ML score meaningfully with the rule-based score. Where category labels are available (banking, SMS, adware, riskware), we additionally train a multi-class head to suggest the likely malware type, which enriches the analyst report. Feature importance and per-sample attribution are retained so that each ML-driven decision can be explained in terms of the specific permissions and APIs that drove it.

4.6 Stage 6 — Hybrid risk scoring

The final risk score will combine the transparent rule-based score, the machine-learning probability, and threat-intelligence corroboration:

$$\text{Risk} \;=\; \text{RuleScore} \;+\; \alpha\,\text{ML} \;+\; \beta\,\text{Intel},$$

normalised to a 0–100 scale and mapped to the severity bands of Table 3. Every contribution is itemised so an analyst can see exactly *why* a sample received its score. Confirmed multi-engine detections raise confidence, while patterns characteristic of benign advertisement-heavy

applications are down-weighted to control false positives. The fusion weights α, β are calibrated on the validation set to optimise the precision/recall trade-off at the chosen operating point, so that neither the ML signal nor external intelligence can override the explainable rule-based evidence on its own.

Score range	Severity
0–20	Low
21–40	Medium
41–70	High
71–100	Critical

Table 3: Risk-score to severity mapping.

4.7 Stage 7 — Generative-AI reasoning and reporting

A generative-AI layer will consume only the *extracted findings* (never the raw application) and produce: an executive summary for non-technical fraud managers, a technical narrative correlating the evidence into a likely attack chain, a list of indicators of compromise, and recommended actions for fraud/SOC teams. The model is prompted to reason critically about the evidence—distinguishing, for instance, benign library usage from genuine malicious intent—so that the report adds analytical value rather than merely restating the findings.

To preserve trust, the layer is *evidence-grounded*: the model is constrained to the structured findings supplied to it and is instructed not to introduce indicators or claims that are not present in that evidence, which suppresses hallucination. The output is requested in a fixed, structured schema so that it can be rendered consistently in the dashboard and exported into the formal report. Sending only distilled findings—rather than the raw application—also keeps the design safe and privacy-preserving, since no potentially malicious artefact is transmitted to the language model. Should the generative service be unavailable, a deterministic template produces an equivalent report from the same findings, so the platform degrades gracefully.

4.8 Stage 8 — Output and integration

The platform will present an analyst dashboard (risk gauge, itemised breakdown, evidence panels, and the AI report) and will export machine-readable artefacts: a formal PDF investigation report and IOC lists suitable for ingestion into blocklists and SOC tooling.

4.9 Platform Architecture

The APKGuard AI platform follows a layered architecture that integrates APK ingestion, static and dynamic analysis engines, feature engineering pipelines, machine-learning classifiers, threat-intelligence services, hybrid risk-scoring mechanisms, and Generative AI reporting components. Each layer contributes explainable evidence that is progressively refined into actionable security intelligence for analysts and fraud investigation teams.

4.10 Technology Stack Map

The implementation of APKGuard AI relies on a modern technology ecosystem spanning frontend development, backend orchestration, malware-analysis frameworks, machine-learning libraries, Generative AI services, databases, cloud infrastructure, monitoring systems, and deployment tooling. This technology stack enables scalable, secure, and maintainable operation of the platform.

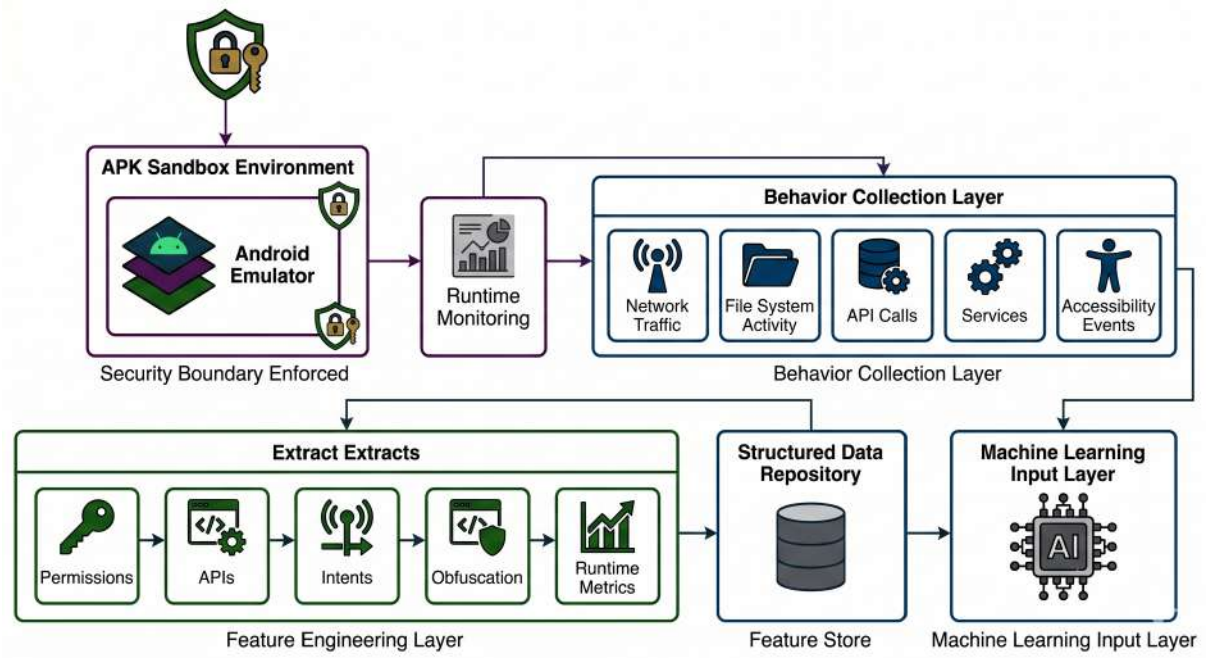


Figure 4: High-level platform architecture of APKGuard AI showing the end-to-end flow from APK submission and analysis through feature engineering, machine-learning classification, hybrid risk scoring, and Generative AI powered reporting.

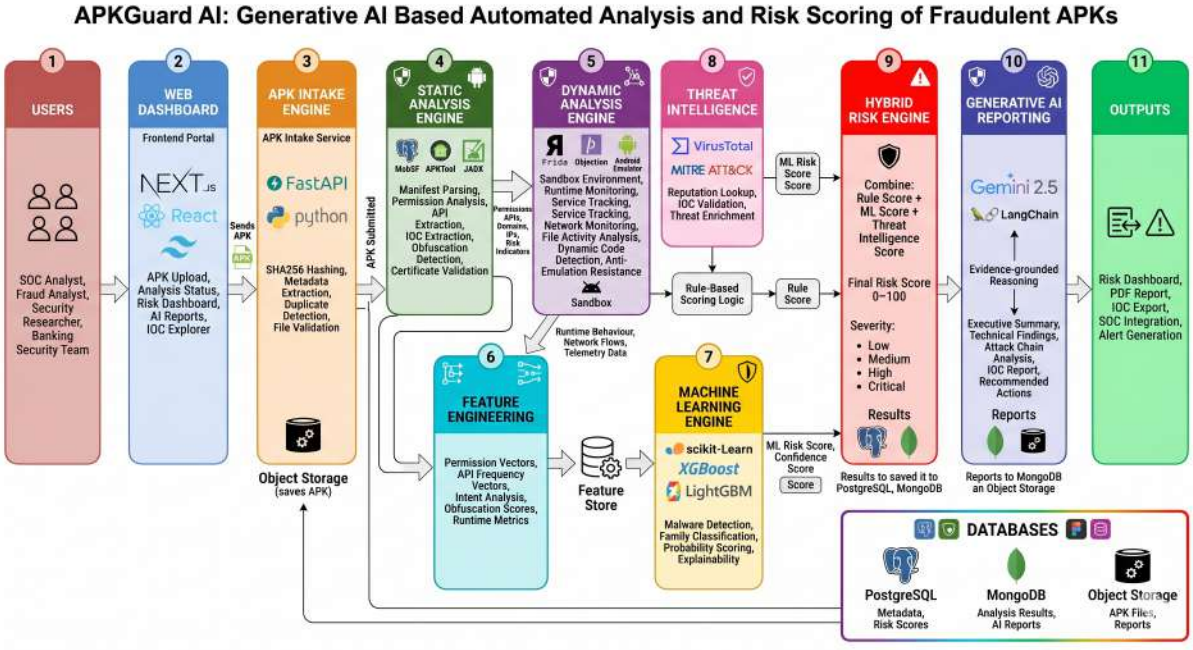


Figure 5: Technology Stack Map illustrating the software frameworks, analysis tools, machine-learning libraries, databases, cloud services, DevOps infrastructure, and monitoring components used to implement the APKGuard AI platform.

5 Evaluation Methodology

5.1 Metrics

Let TP, FP, TN, FN denote true/false positives/negatives. We report

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}, \quad \text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}, \quad \text{F1} = \frac{2\,\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}},$$

together with ROC-AUC and the false-positive rate FPR = FP/(FP + TN). We emphasise precision/recall and FPR over raw accuracy, since class imbalance makes accuracy misleading.

5.2 Protocols

- **In-distribution:** stratified hold-out and cross-validation on each corpus.
- **Temporal (drift):** train on earlier samples, test on later ones, to estimate deployment performance as malware evolves.
- **Cross-dataset:** train on one corpus, evaluate on another, to measure generalisation.
- **False-positive focus:** dedicated evaluation against a curated set of modern benign applications, since over-flagging legitimate apps erodes analyst trust.
- **Explainability checks:** verify that high-scoring decisions are supported by human-interpretable evidence and that feature importances are consistent with domain knowledge.

5.3 Baselines and ablations

We compare the hybrid system against (i) rule-based scoring alone and (ii) the ML model alone, and we ablate feature groups (permissions only, +API, +dynamic) to quantify the marginal value of each evidence source and of dynamic analysis.

6 Threats to Validity and Limitations

- **Concept drift.** Models trained on historical data degrade over time; temporal evaluation and periodic retraining mitigate but do not eliminate this.
- **Evasion.** Heavily packed samples, native-code logic, and anti-analysis checks can resist both static and dynamic inspection; evasion-resistance reduces but cannot fully remove this risk.
- **Unavailable infrastructure.** Samples whose payloads are fetched from now-defunct C2 servers may not exhibit full behaviour dynamically; in such cases the static evidence and captured indicators remain the authoritative signal.
- **Label noise.** Multi-engine verdicts disagree; threshold policies and reconciliation mitigate this, but residual uncertainty remains.
- **Dataset bias.** Public corpora over-represent certain families and eras; combining datasets with fresh sampling reduces, but does not remove, bias.

7 Ethical and Safety Considerations

All potentially malicious samples will be handled and executed only within isolated environments using no personal accounts or networks. The system is designed for *defensive* purposes—first-level triage and investigation support for banking fraud and SOC teams. Reporting will be

faithful to the evidence, and limitations (e.g. concept drift, sandbox evasion, payloads gated behind unavailable infrastructure) will be stated transparently rather than overstated.

Source Code Repository

`github.com/IntrosWorld/malware-apk-detect`

APKGuard AI Prototype

The APKGuard AI prototype demonstrates the complete end-to-end workflow of the proposed platform, integrating APK ingestion, static and dynamic malware analysis, feature engineering, machine-learning driven risk assessment, hybrid risk scoring, threat-intelligence enrichment, and Generative AI powered investigation reporting.

The platform enables analysts to upload suspicious Android applications, inspect extracted indicators of compromise, visualize explainable risk scores, and generate evidence-grounded security reports for fraud investigation and SOC operations.

Open Source • Research Prototype • Production Ready

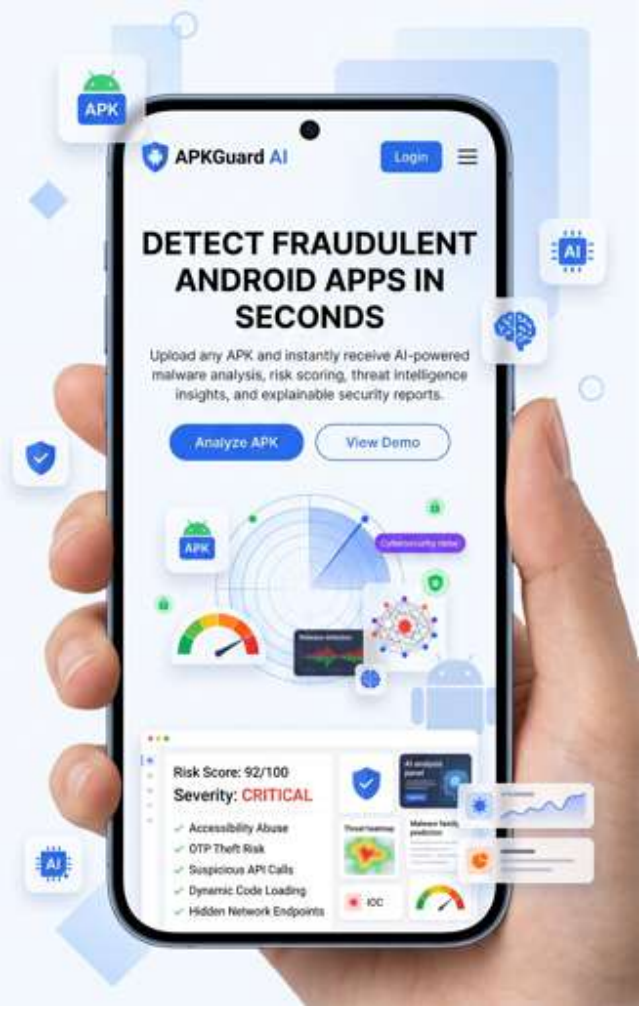


Figure 6: APKGuard AI implementation and prototype demonstration. The GitHub repository provides the complete source code, while the prototype showcases automated APK analysis, threat detection, explainable risk scoring, and Generative AI powered reporting capabilities.

8 Initial Prototype Photos

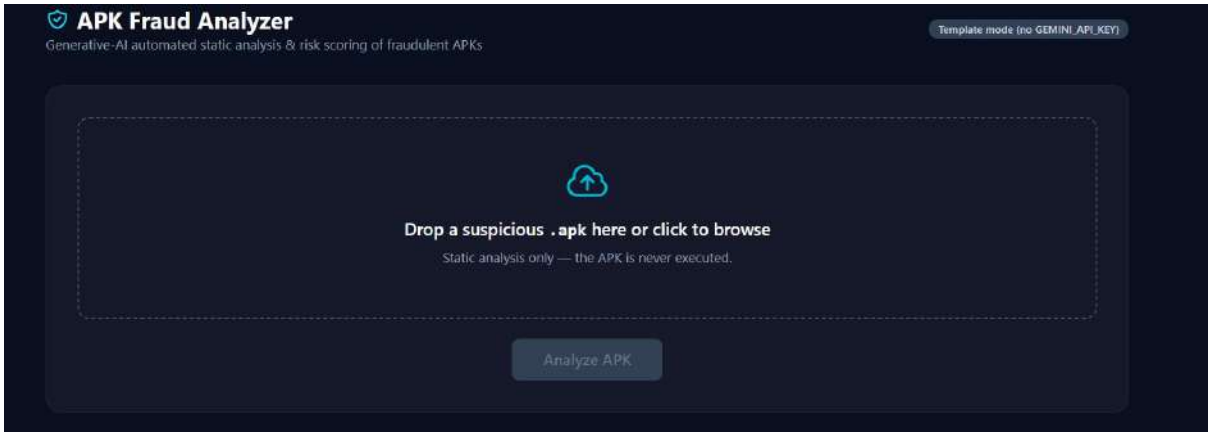


Figure 7: Prototype View 1

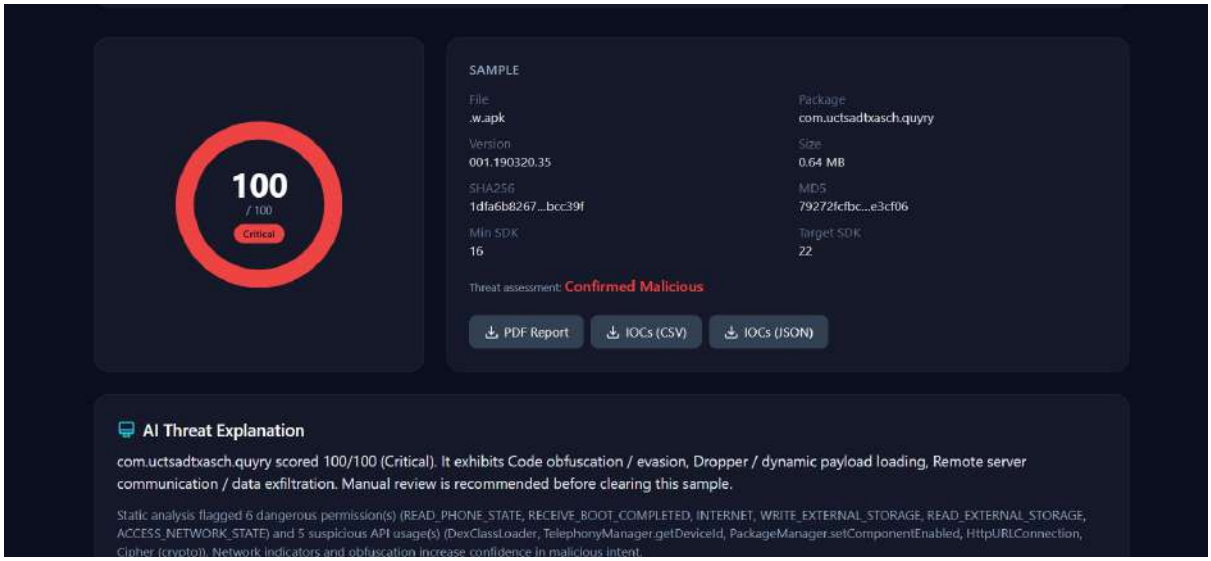


Figure 8: Prototype View 2



Figure 9: Prototype View 3

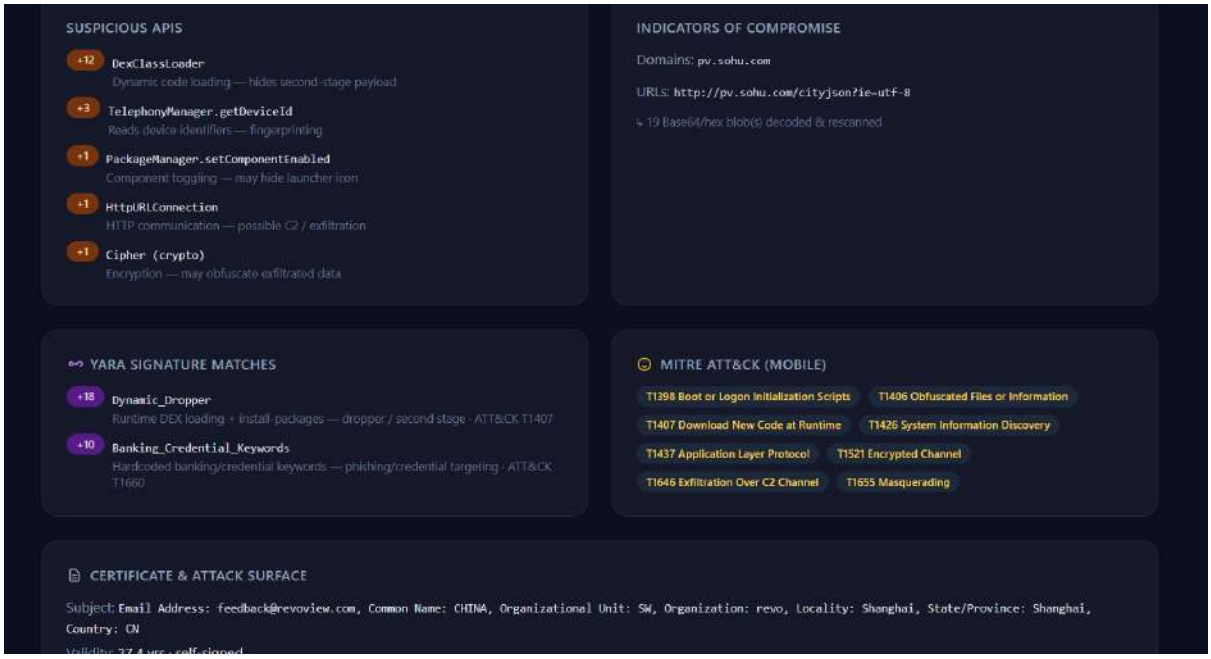


Figure 10: Prototype View 4

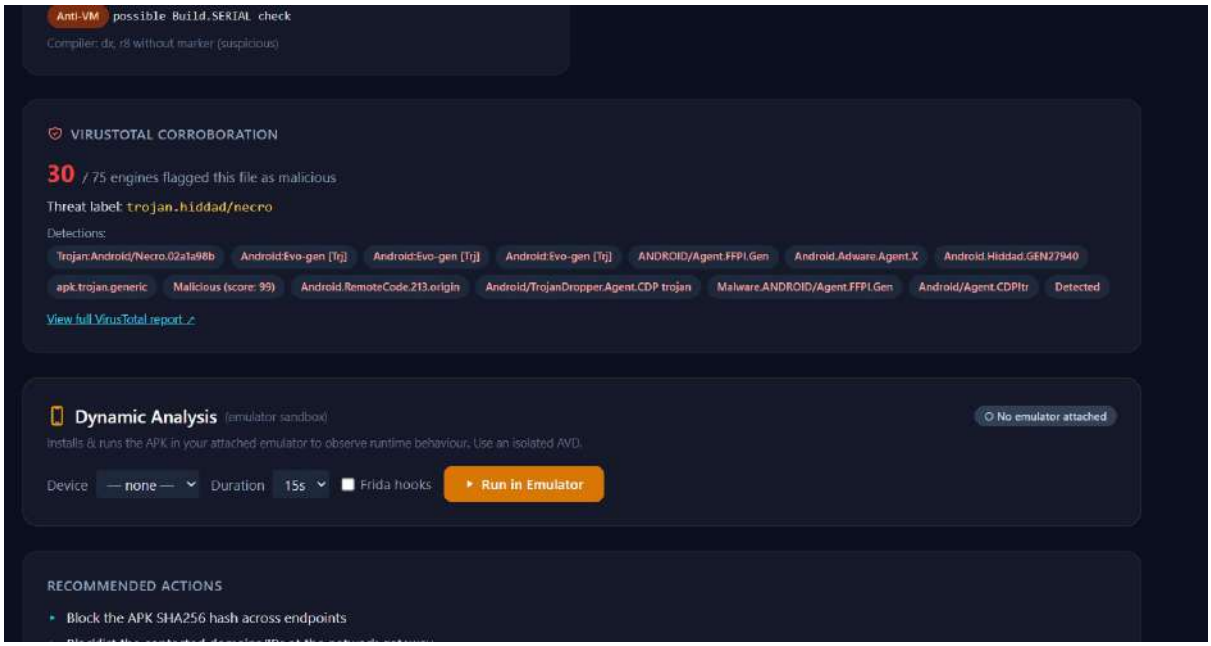


Figure 11: Prototype View 5